

UC Security Model

CS714: Secure Computation

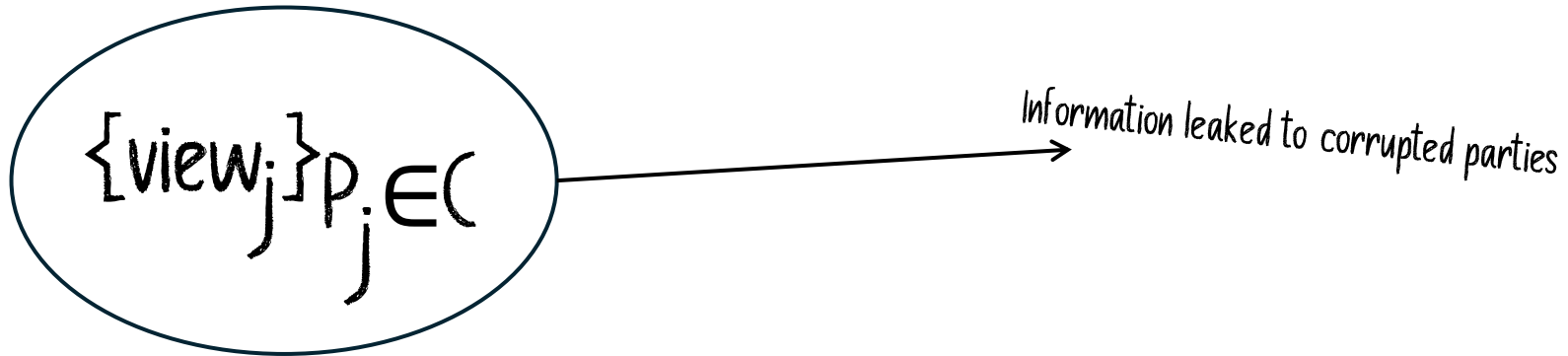
Logistics

- Thanks for your feedback!
- Optional Recap Class on Saturday
 - No new material will be covered (this is purely optional)

Defining Privacy

For each corrupted party P_j define view_j the entire view of the party P_j

$C \subseteq \{P_1, \dots, P_n\}$ is the set of corrupted parties



$$f(x_1, x_2, x_3) = (x_1 x_2 x_3, x_2, x_3)$$

The diagram illustrates the mapping of inputs to parties and the resulting outputs of a function f . The function is defined as $f(x_1, x_2, x_3) = (x_1 x_2 x_3, x_2, x_3)$. Arrows point from each input variable to a party: x_1 to P_1 , x_2 to P_2 , and x_3 to P_3 . The output of the function is shown as a tuple of three values. Arrows point from these output values to the parties: the first output value $x_1 x_2 x_3$ points to P_1 , the second output value x_2 points to P_2 , and the third output value x_3 points to P_3 .

Instead of real input x_1 the party P_1 can use $x'_1 = 1$

P_1 can then learn $1 \cdot x_2 \cdot x_3$

The adversary can influence the output

This does not necessarily violate security (parties are allowed to choose arbitrary inputs)

A protocol proven secure in the UC model remains secure regardless of the context in which it is used.

Allowed vs Leaked Information

A protocol is private if the leaked values contain no more information than the allowed values.

Computational View

A protocol is private if the leaked values can be computed efficiently from the allowed values.

The Idea of a Simulator

An efficient algorithm that generates the leaked values from the allowed values

There exists an efficient simulator S such that:

$S(\{x_j, y_j\}_{j \in C})$ & $\{\text{view}_j\}_{j \in C}$ have the same distribution

The Idea of a Simulator

The adversary's view can be simulated using only:

corrupted parties inputs

corrupted parties outputs

Behavioral Equivalence

When do we say that two interactive systems are behaviorally equivalent?

When the two systems get the same output on the same output ports, and get the same inputs on the same input ports

We are not interested in the *internal* structure of the Interactive Systems

Security via Behavioral Equivalence

UC security compares:

Real Protocol

Ideal Functionality

The real protocol must be behaviorally equivalent to the ideal functionality

Ideal Functionality

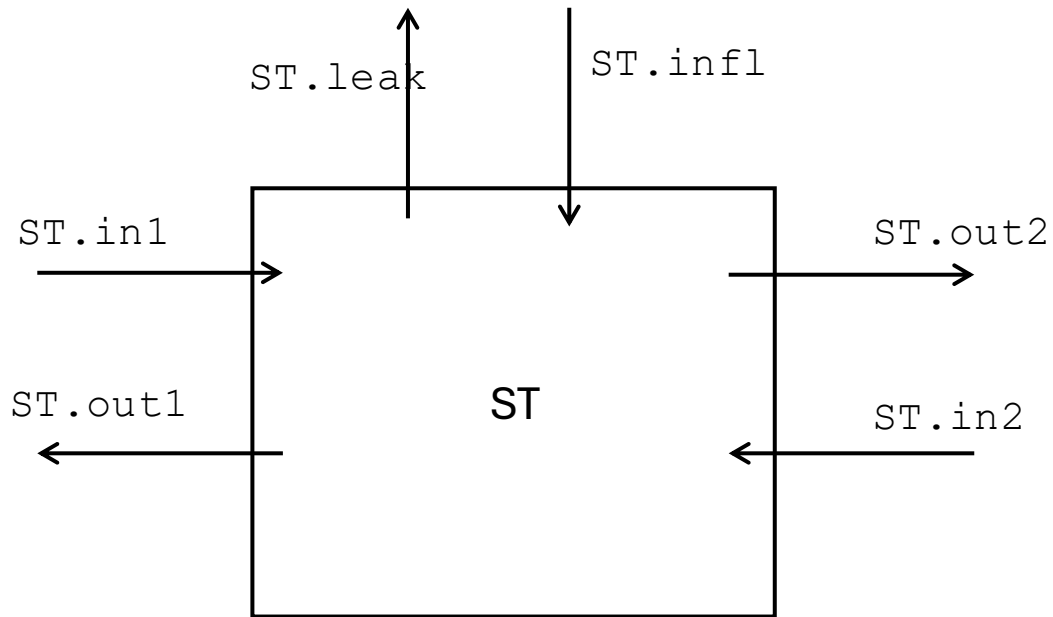
An ideal functionality F specifies how the protocol must behave.

The protocol may leak only *allowed information*.



Any information that can be computed from the inputs and outputs of the corrupt parties

Ideal Functionality: Secure Transfer



On input (mid, j, m) on $ST.in_i$
output $(mid, i, j, |m|)$ on $ST.leak$
store (mid, i, j, m)

Ignore any later input of the form (mid, j, m) on $ST.in_i$

On input $(deliver, mid, i, j)$ on $ST.infl$

delete: (mid, i, j, m)

output: (mid, i, j, m) on $ST.out_j$

Modeling Corruption

(passive corruption is) $ST.infl$

The ideal internal state of *party i*.

This is defined as the previous messages of *party i*

And all the messages that are in the queue.

(active corruption is) $ST.infl$

The ideal functionality records that *party i* is corrupt

Ignores inputs on $ST.in_i$ stops giving outputs on: $ST.out_i$

Instead, whenever it gets (input, i, x) on $ST.infl$ it behaves exactly as input came from: $ST.in_i$

Whenever ST is about to output a value y on $ST.out_i$ It instead outputs (output, i, y) on $ST.leak$

Protocols

$\Pi = \{P_1, \dots, P_n\}$ be a protocol with resource name R

Let R be the ideal functionality for the resource R

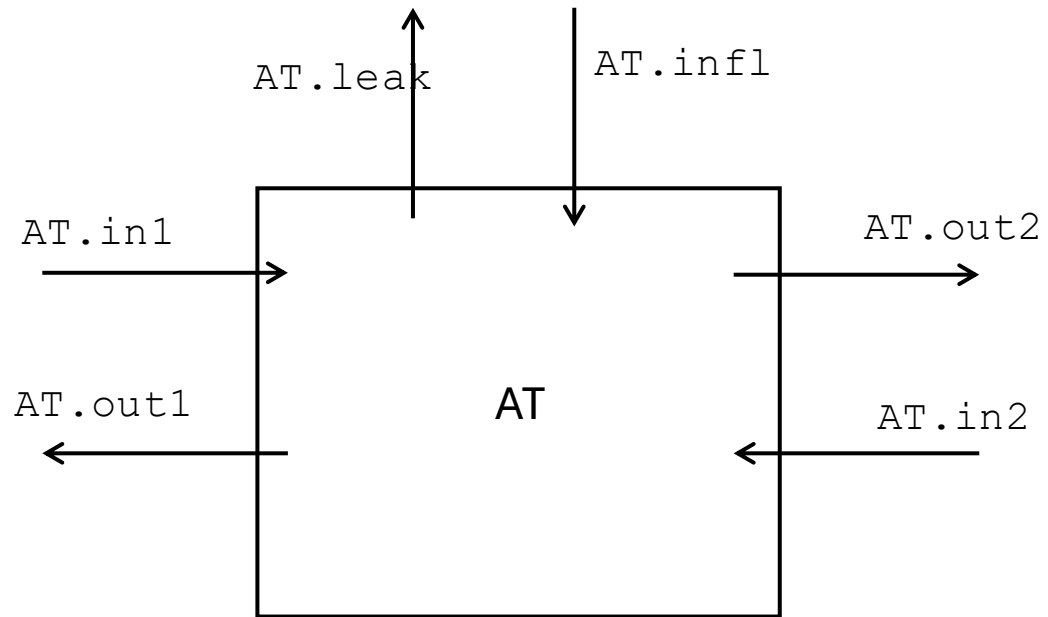
$\Pi \diamond R$ a protocol using resource R

Ideal functionalities can play two roles:

- Specifications of intended behavior

- It can also serve as a network resource (a way for them to communicate)!

Protocols



On input $(mid, j, AT.in_i, m)$ on $AT.in_i$
 output (mid, i, j, m) on $AT.out_i$
 store (mid, i, j, m)

Ignore any later input of the form (mid, j, m) on $AT.in_i$

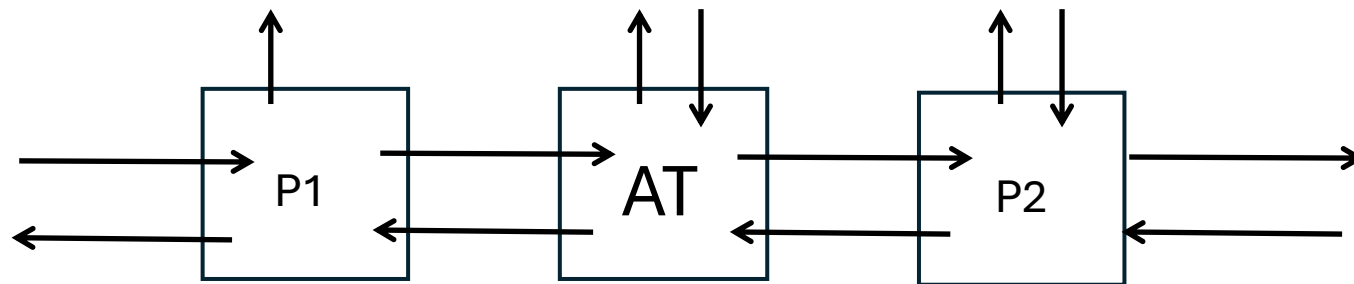
On input $(deliver, mid, i, j)$ on $AT.infl$

delete: (mid, i, j, m)
 output: (mid, i, j, m) on $AT.out_j$

Protocols

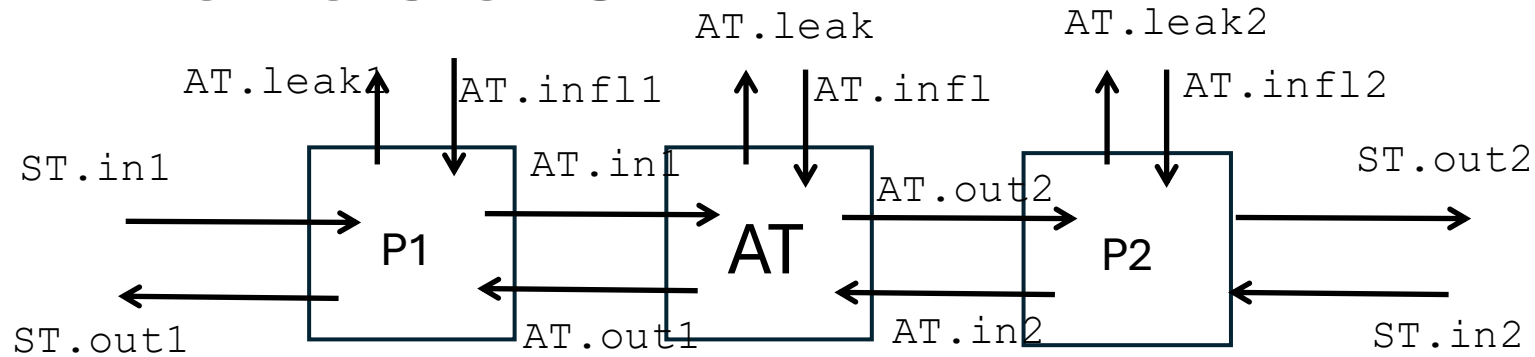
$$\Pi = \{P_1, P_2, \dots, P_n\}$$

uses an ideal functionality to implement another ideal functionality



resource

Protocols



$$\Pi_{ST} \diamond F_{AT}$$

What does this mean?

Agent P1

On input (mid,2,m) on ST.in1, output (mid,2, hello) on AT.in1.

When receiving (mid, 2, ek) on AT.out1, sample encryption $c = E_{\{ek\}}(m)$

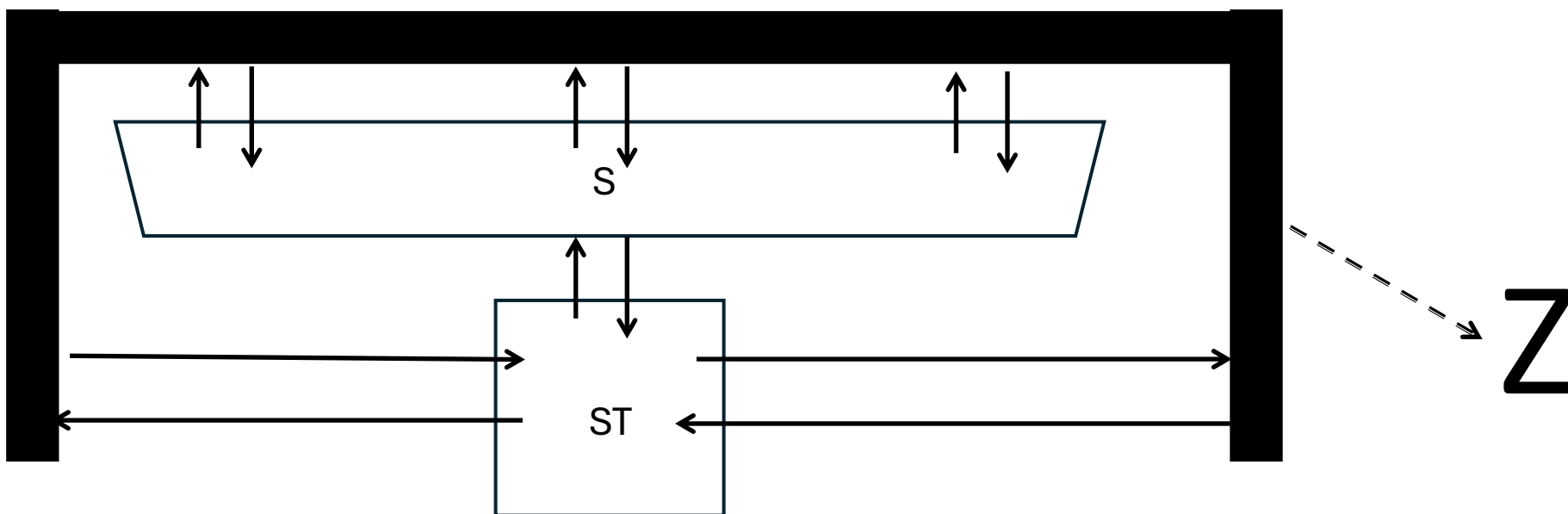
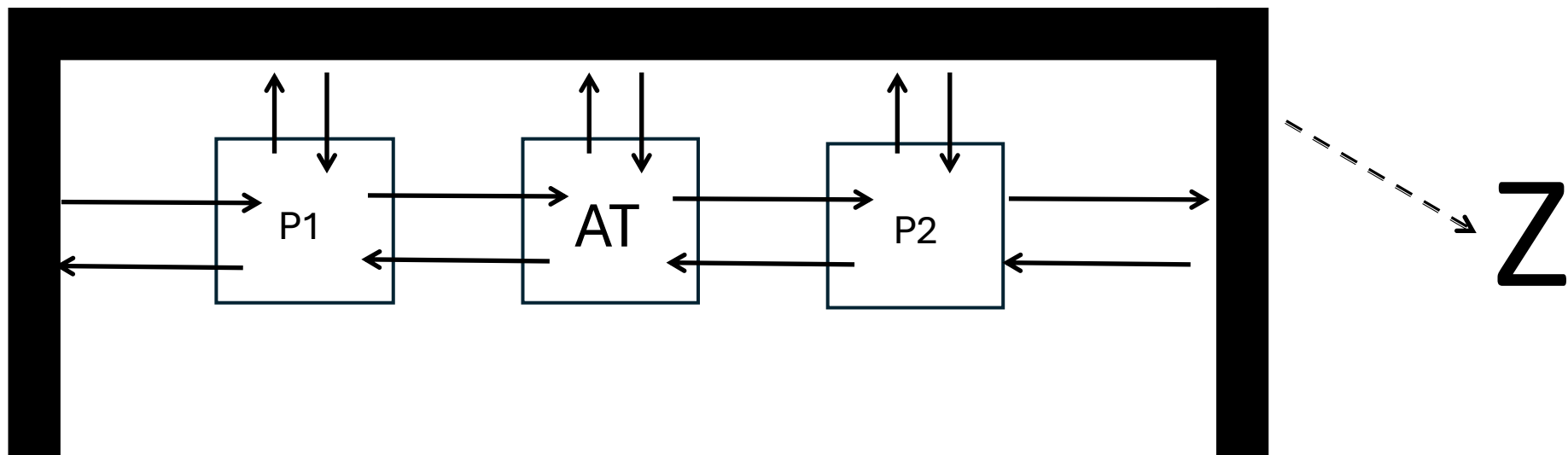
and output (mid,2,c) on AT.in1.

Agent P2

On input (mid,1, hello) on AT.out_2,
generate key pair (ek,dk) and send (mid,1,ek).

On input (mid,1,c) on AT.out_2, compute $m = D_{\{dk\}}(c)$

and output (mid,1,m) on ST.out_2.



How does Environment Z behave?

Picks some message m and inputs $(mid, 2, m)$ on $ST.in1$

Then it sees $(mid, 1, 2, \text{hello})$ on $AT.leak$ and inputs $(deliver, mid, 1, 2)$

Then it sees $(mid, 2, 1, ek)$ on $AT.leak$ and inputs $(deliver, mid, 2, 1)$

Then it sees $(mid, 1, 2, c'')$ on $AT.leak$ and inputs $(deliver, mid, 1, 2)$

Then it sees $(mid, 1, m'')$ on $ST.out2$

Standalone Security

The protocol is secure *when run alone*.

Adversary interacts with *only this execution*.

Proof compares the real vs ideal world for a single run

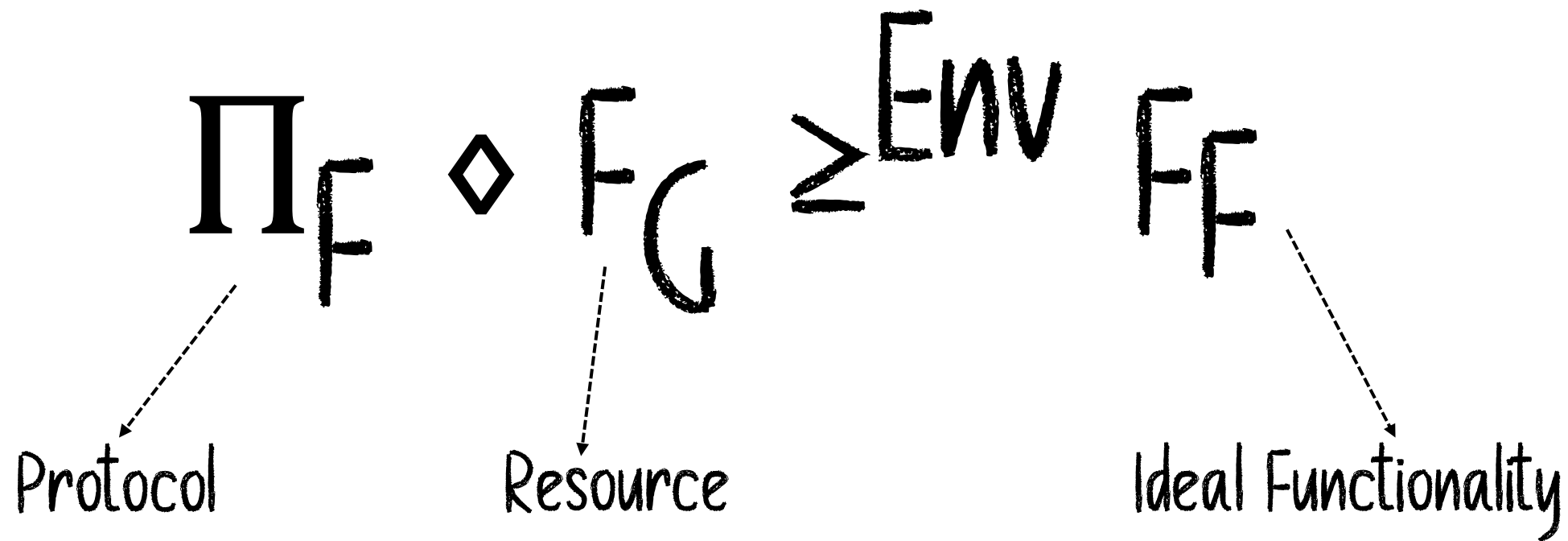
Composed Security

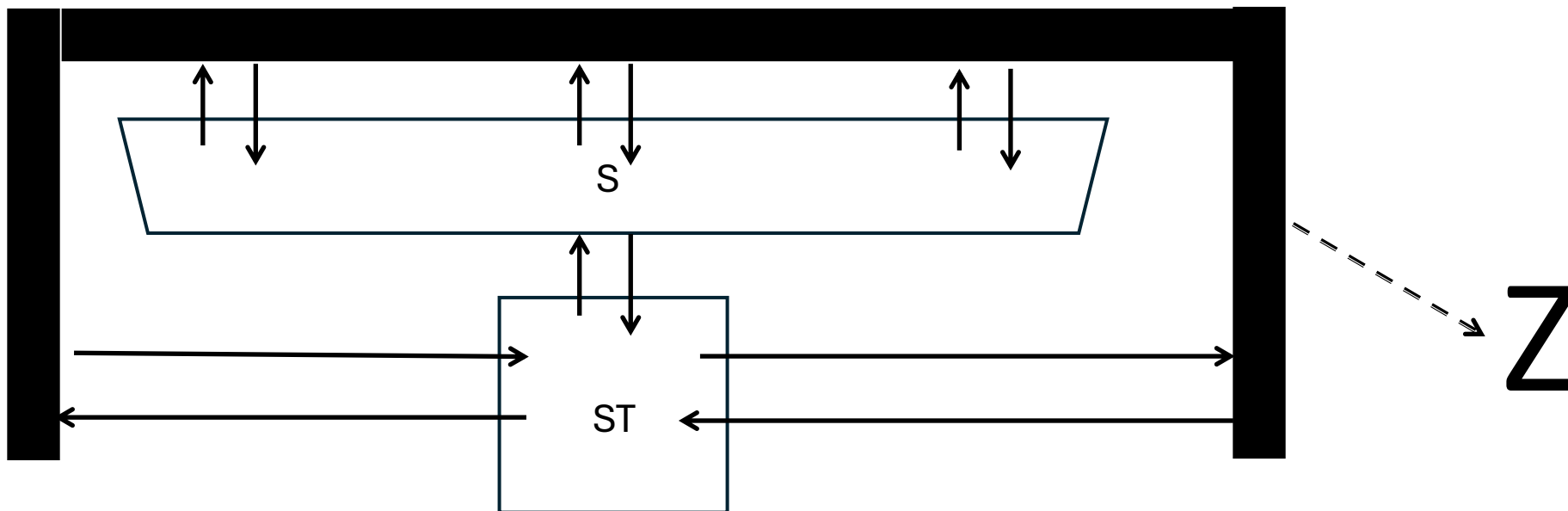
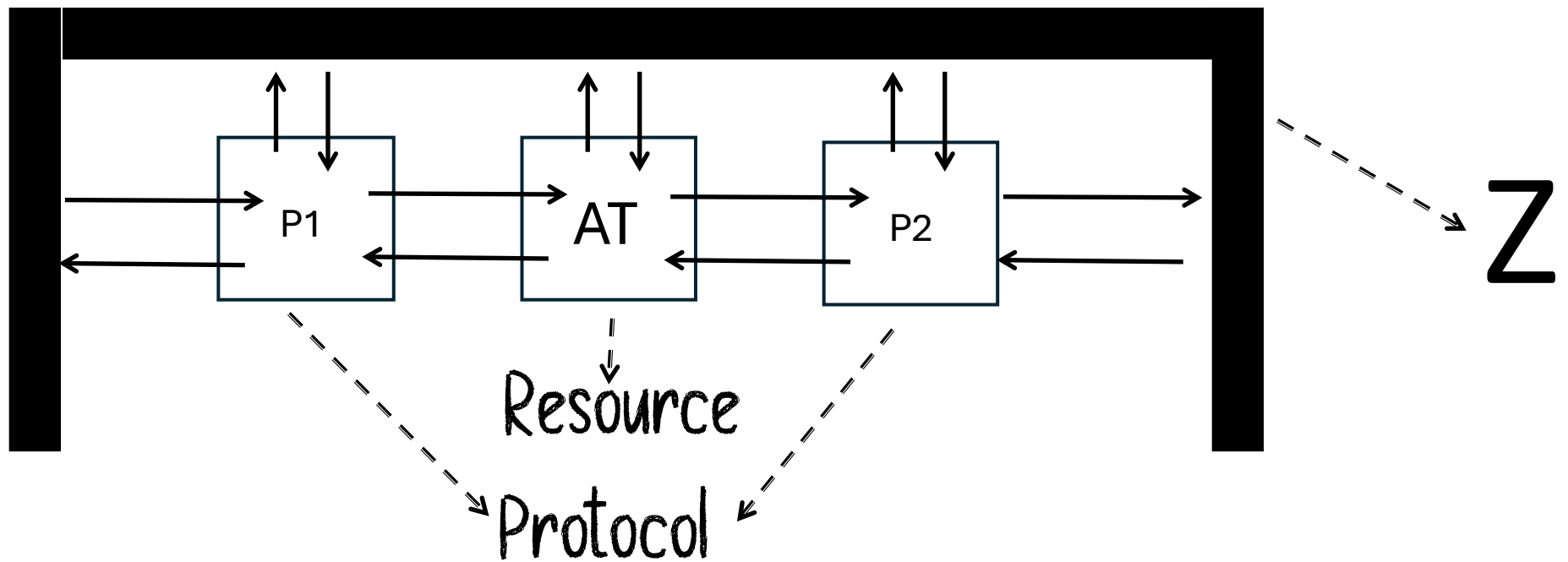
Protocol remains secure *even when composed with arbitrary other protocols*

An adversary can interleave many executions

The environment can interact during execution

As secure as!



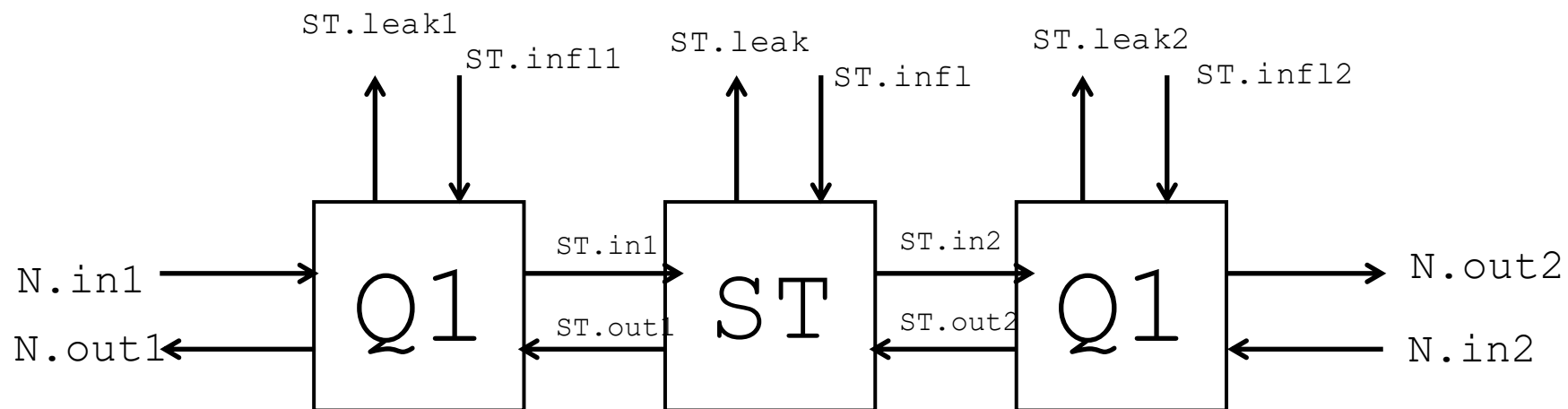
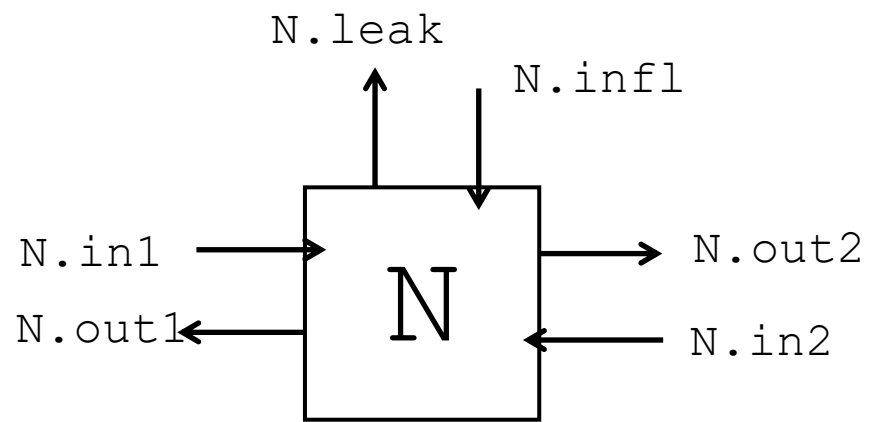


Composed Protocols

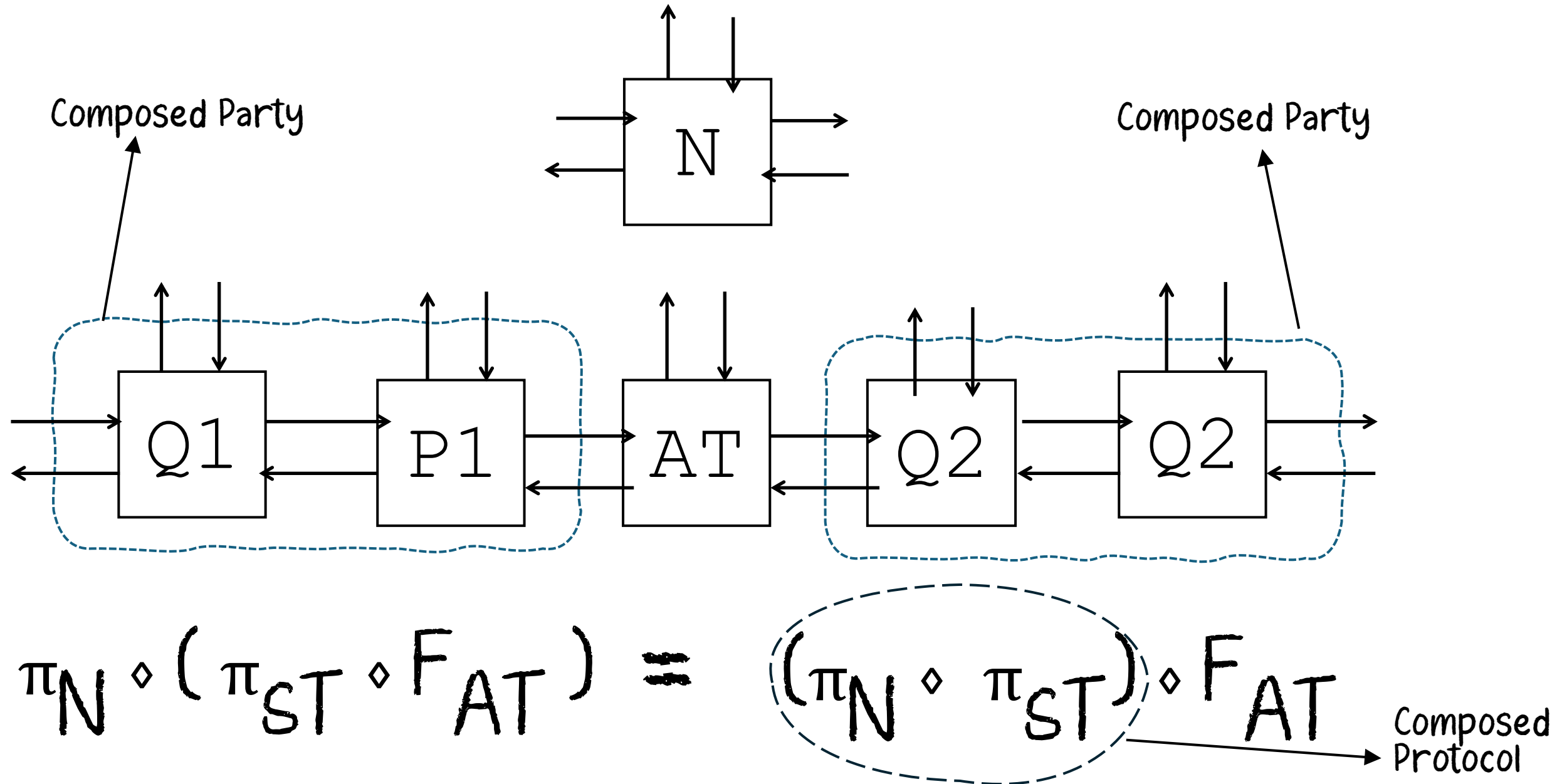
$\Pi_{ST} \diamond F_{AT}$ implements F_{ST} with computational security, then:

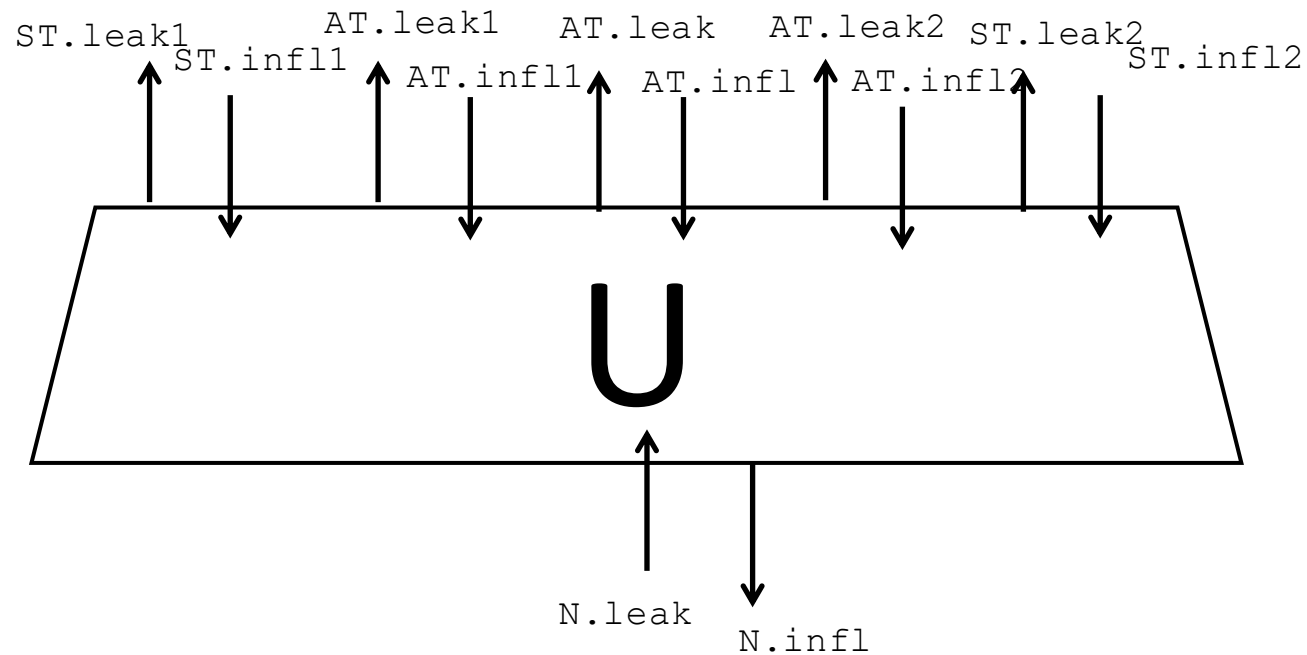
F_{ST} can be securely replaced by $\Pi_{ST} \diamond F_{AT}$

in any polynomial-time protocol



$$\pi_N \diamond F_{ST}$$





$$\pi_N \diamond \pi_{ST} \diamond F_{AT}$$

Security of Composed Protocols:

Existence of a simulator

U

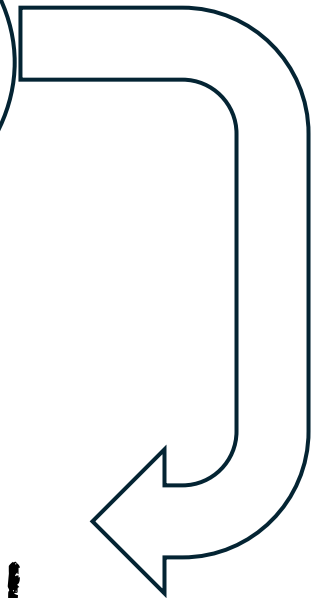
UC Theorem

Let Env be a set of polynomial-time environments

$$\pi_N \circ F_{ST} \geq^{\text{Env}} F_N$$

$$\pi_{ST} \circ F_{AT} \geq^{\text{Env}} F_{ST}$$

$$\pi_N \circ \pi_{ST} \circ F_{AT} \geq^{\text{Env}} F_N$$



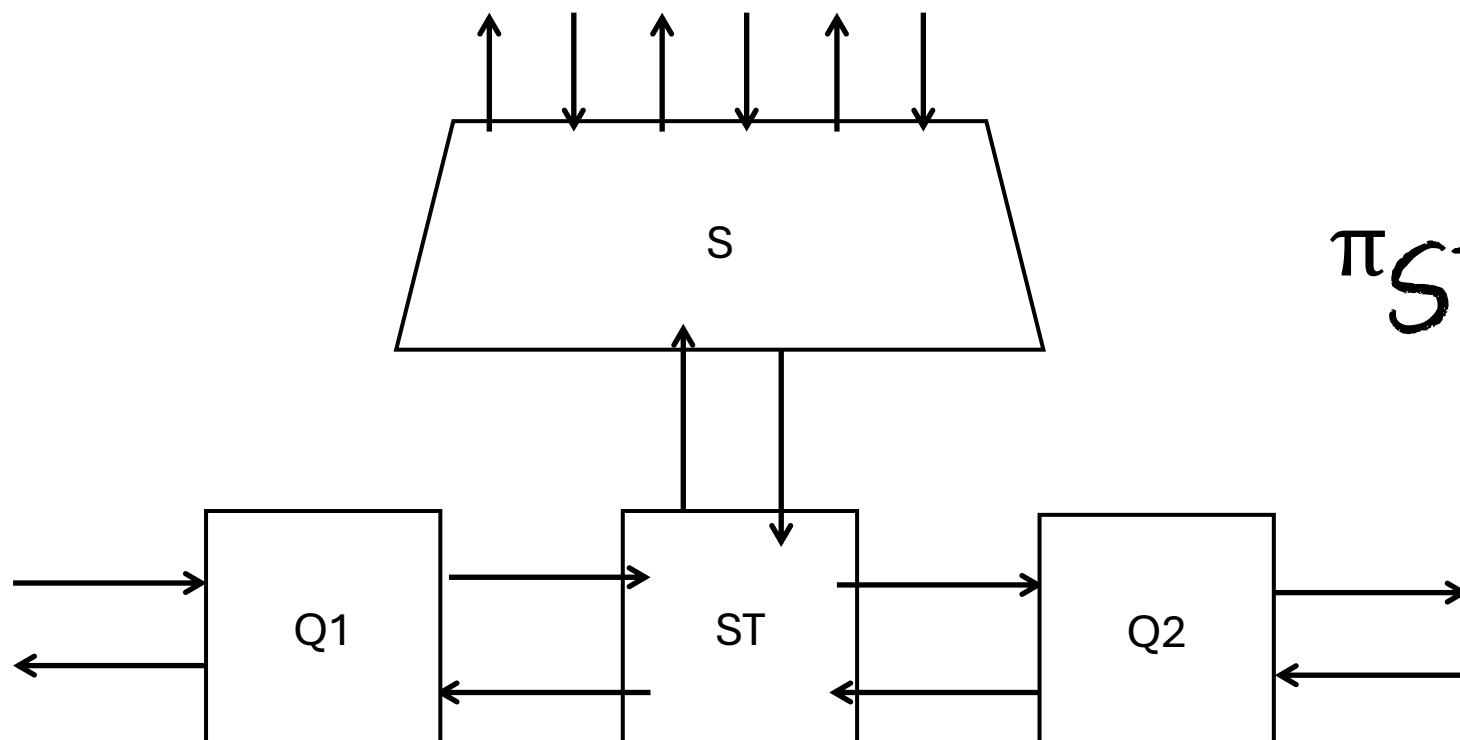
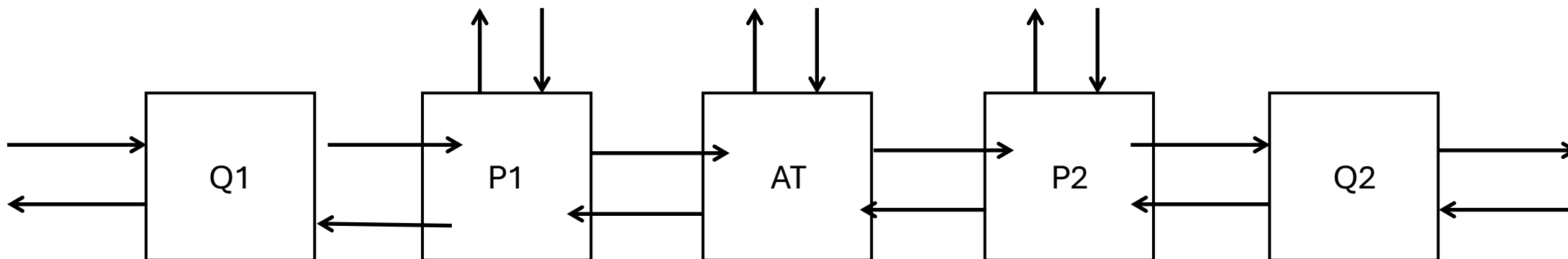
UC Theorem

Let Env be a set of polynomial-time environments (INTUITION)

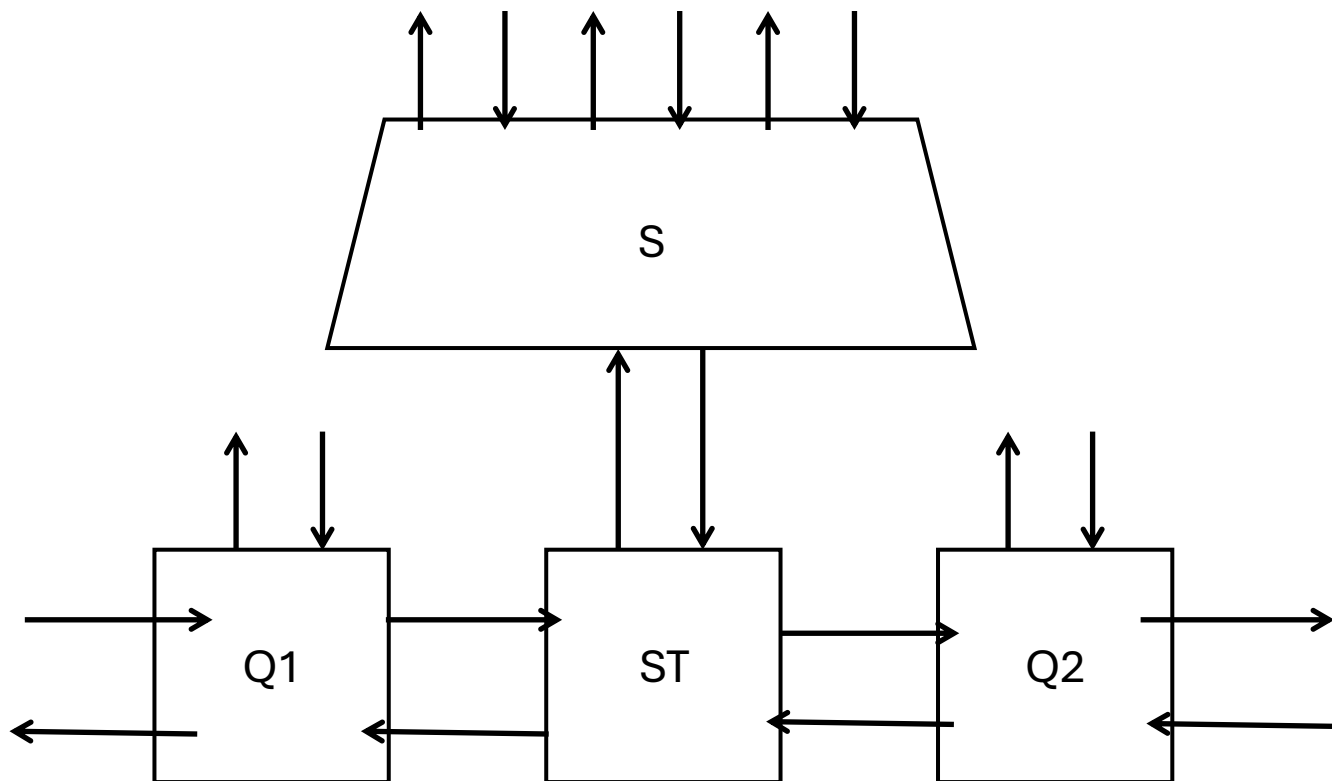
$$\pi_N + F_{ST} \geq^{\text{Env}} F_N$$

$$\pi_{ST} + F_{AT} \geq^{\text{Env}} F_{ST}$$

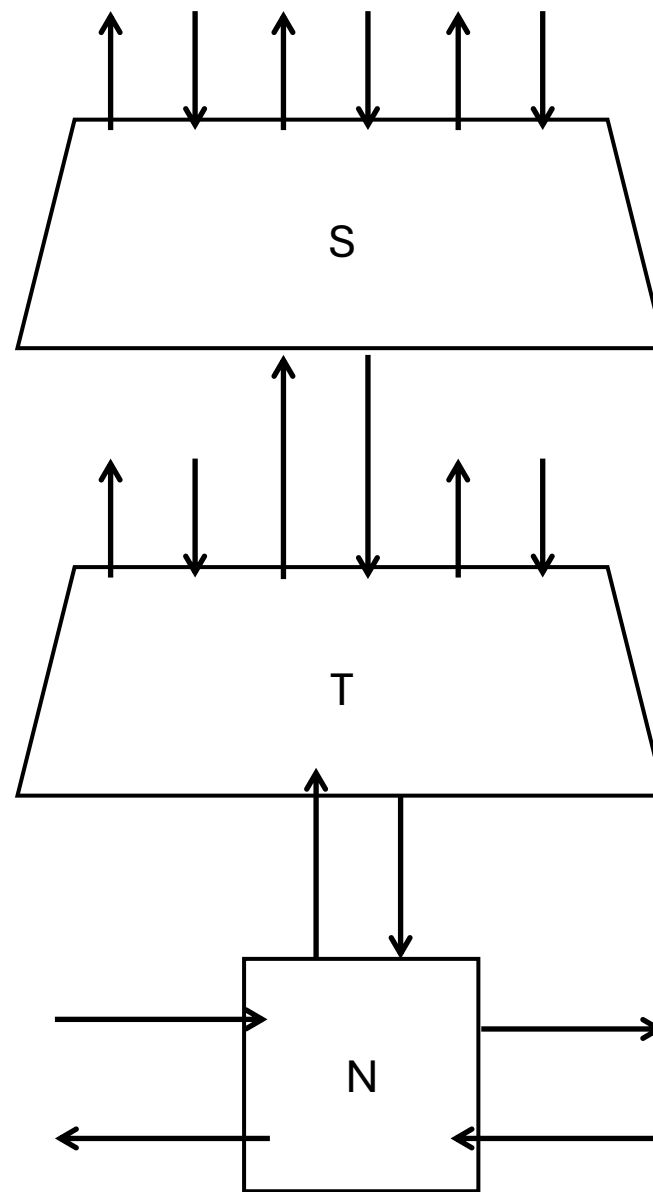
$$\pi_N + \pi_{ST} + F_{AT} \geq^{\text{Env}} F_N$$



$$\pi_{ST} \diamond F_{AT} \geq^{\text{Env}} F_{ST}$$



$$\pi_N \circ F_{ST} \geq^{Env} F_N$$

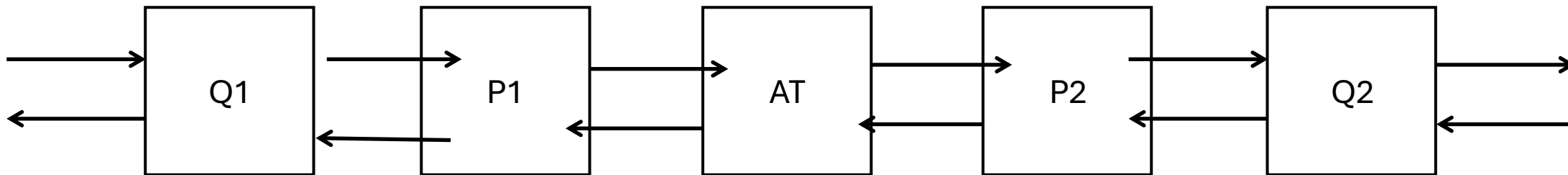


IS0 cannot be told apart from IS1

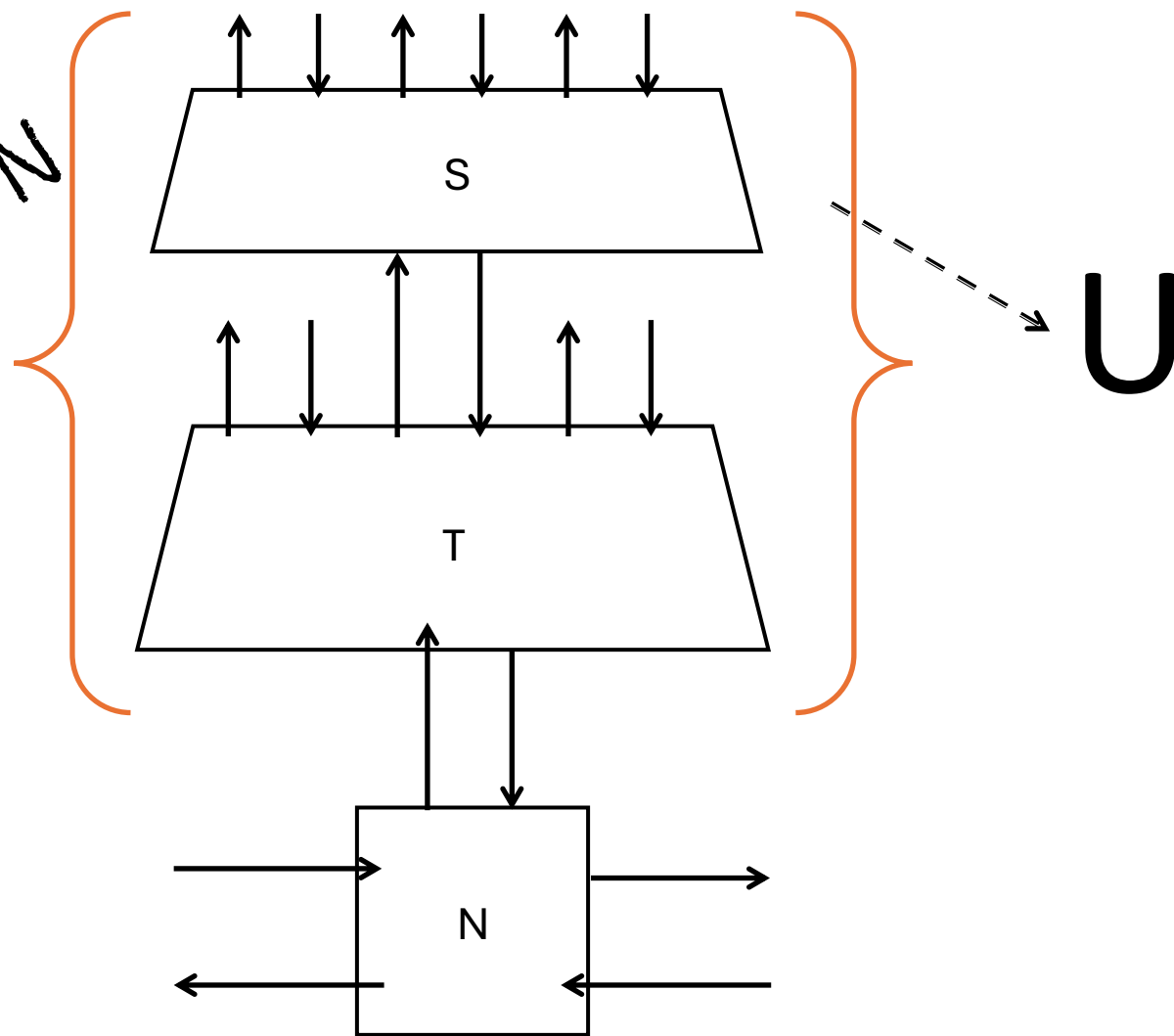
IS1 cannot be told apart from IS2

From the transitivity property, we have:

IS0 cannot be told apart from IS2



$$\pi_N + \pi_{ST} + F_{AT} \geq_{\text{Env}} F_N$$



Is there any problem with the proof that we presented?

We are assuming that: $\Pi_N \diamond Z$ is an environment

We let **Pro** be the set of simple and composed protocols in which all simple parties follow the rules for clocked entities and are recursive in polytime.

If $\pi_F \in \text{Pro}$ is a protocol with the name F and resource name R
And $\pi_R \in \text{Pro}$ is a protocol with protocol name R and $\pi_F \diamond \pi_R \neq \perp$

Then: $\pi_F \diamond \pi_R \in \text{Pro}$

Defn

We let SIM be the set of interactive systems S that are a simulator for some protocol

There exists a protocol Π_F (composed or simple) with protocol name F and an ideal functionality F_F

With the name F such that $F_F \diamond S$ have the same special ports

S follows the rules for clocked entities and is recursive polytime

S is corruption preserving

S is clock preserving

Lemma

If $S \in \text{Sim}$ and $T \in \text{Sim}$

and T has two open special ports that connect it to the ideal functionality ports of S and $S \diamond T \neq \perp$

then: $S \diamond T \in \text{Sim}$

Defn

We say that Env is an environment class if the following hold true:

Each $Z \in \text{Env}$ has the open port structure of the environment for some simple or complex protocol

For all $Z \in \text{Env}$ and all $\Pi \in \text{Pro}$ where $\Pi \diamond Z \neq \perp$ and the protocol ports of Π
connect to the protocol ports of Z
it holds that: $\Pi \diamond Z \in \text{Env}$

For all $Z \in \text{Env}$ and all $S \in \text{Sim}$ where $S \diamond Z \neq \perp$ and the simulation ports of S
connect to the special ports of Z
it holds that: $S \diamond Z \in \text{Env}$

UC Theorem (on Board)

Revision of Chapter 4 on Saturday, 10:30 AM, KD 101
(This is optional; no new material)

Quiz 2 will be on Tuesday, 7-April-2026, Syllabus:
Chapter 4

Adversaries and Their Powers

Threshold Security

Adaptive Security vs. Static Security

Active Security vs. Passive Security

Unconditional Security vs. Computational Security

Synchronous vs. Asynchronous

Threshold Security

Remember the CEPS Protocol:

The protocol is secure against $t < n/2$ corrupted parties.

This is called Threshold Security!

Adaptive Security vs. Static Security

Environments in Env are allowed to corrupt the parties whenever they desire

Environments in Env are allowed to corrupt the parties before the protocol begins

Regular Encryption

$$c = \text{Enc}_{pk}(m; r)$$

c is bound to a unique message m

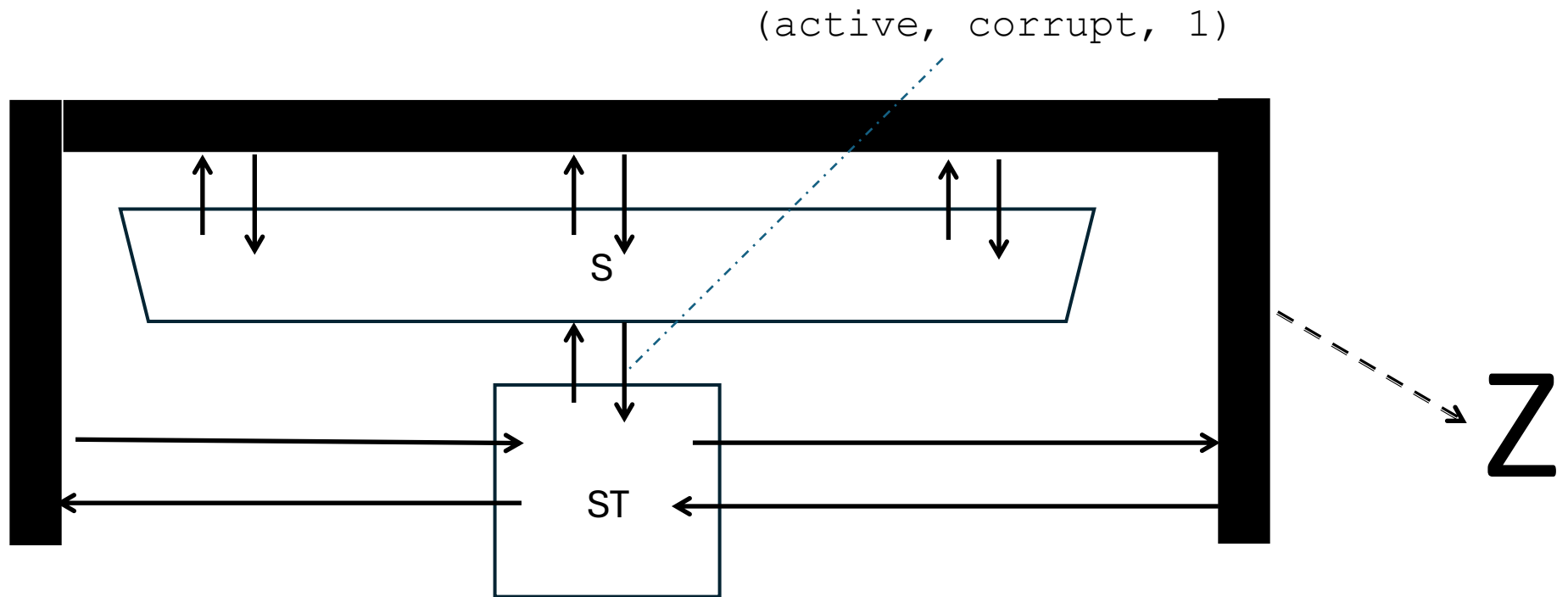
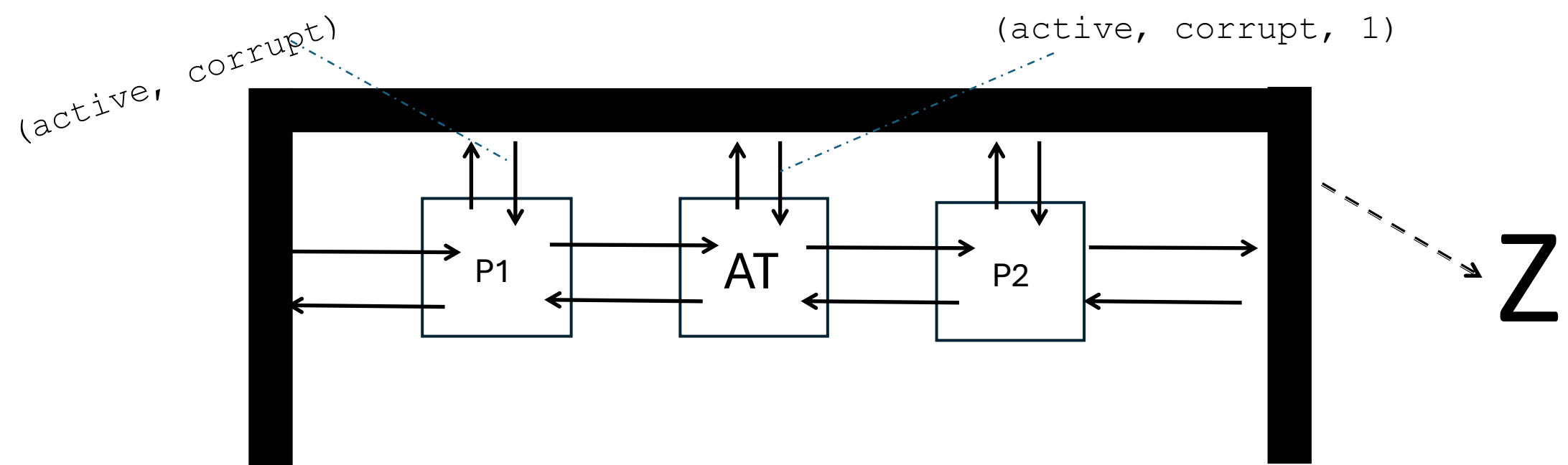
Non-Committing Encryption

Generate a fake public key pk

Generate a ciphertext c *without knowing the message*

Later, when given any message m , produce randomness r such that:

$$c = \text{Enc}_{pk}(m; r)$$



S is now allowed to send: $(\text{send}, 1, 2, \text{mid}, m')$ on ST.infl at any point

In response F_{ST} stores $(\text{mid}, 1, 2, m')$ as if it came from ST.inl

Then, S can send $(\text{deliver}, \text{mid}, 1, 2)$ on ST.infl

In response F_{ST} outputs $(\text{mid}, 1, m')$ on ST.out2

Thus, S can output any message of his choice on $\text{ST.out1}/\text{ST.out2}$

So, how will the Simulation work?

What if P2 is corrupted?

Whenever S is activated, it checks whether it received some $(mid, 1, 2, |m|)$
on ST.leak

If yes: It inputs $(deliver, mid, 1, 2)$ on ST.infl

In response, F_{ST} outputs $(mid, 1, m)$ on ST.leak!

WHY?

The argument here was straightforward because we considered only two parties.

What can go wrong in a three-party setting?

Setting:

P1: Sender

P2: Receiver (corrupted/controlled by adversary Z)

P3: Honest party

Adversary's Goal

The environment Z tries to create a fake execution history.

It makes P2 output: $(mid, 3, m')$

Interpreted as: "P2P received message m' from P3"

But in Reality

P3 **never sent** any such message

No corresponding input: $(mid, 2, m')$ on $ST.in3$ $(mid, 2, m')$ on $ST.in3$

Why This Breaks Simulation

Ideal functionality FST enforces:

A message from P3 can be delivered **only** if P3 actually sent it

Simulator's Problem

To match the real-world view, the simulator must make: $FST \rightarrow (mid, 3, m)$

But this is impossible because FST never received input from P3P3

Conclusion

- **Real world:** Fake message appears
- **Ideal world:** Such a message cannot be generated

How is this attack prevented?

Authenticated communication

(i.e., messages cannot be forged as coming from another party)

What changes in Adaptive Security?

Two parties P1, P2, using an encryption scheme like RSA

Environment Z sends a random message, $m \in \{0,1\}^k$

P1 encrypts: $c = \text{Enc}_{pk}(m; r)$

After seeing the ciphertext, the adversary adaptively corrupts P1

What changes in Adaptive Security?

Now consider the simulation

Before corruption, S **does NOT** know m
(because P_1 is still honest)

Produce a **fake** pk and a **fake** ciphertext c

S commits to some (pk, c)

Later:

Adversary corrupts P1

Now S learns the real m from: F_{ST}

Now the problem: S must produce randomness r such that:

$$c = \text{Enc}_{pk}(m; r)$$

What is the solution then?

Active Security vs Passive Security

The environment takes complete control

Also called semi-honest setting

Unconditional Security vs Passive Security

Consensus Broadcast vs Point-to-Point Communication

Suppose a protocol requires a player broadcast a message to all the players.

It is not just enough to ask the player to send the same message to all the players.

Why?

We need a consensus agreement (Byzantine Agreement)

Synchronous vs Asynchronous

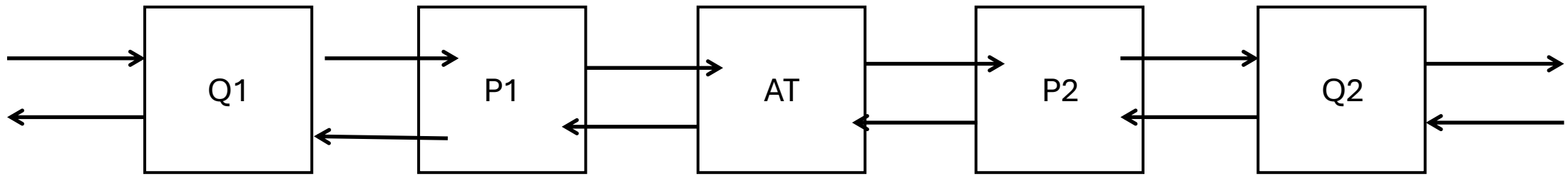
Synchronous Communication

Inward Clocking

Allows a party to send the messages to the ideal functionality in that round

Outward Clocking

Allows a party to receive the messages from the ideal functionality in that round

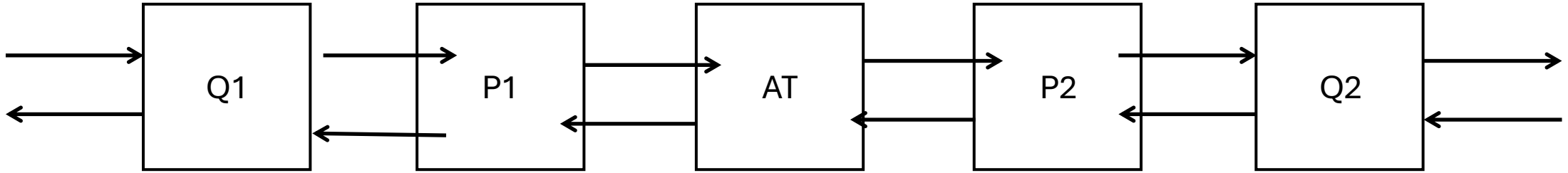


We say that Z is a synchronous environment if it proceeds in rounds, where in each round it behaves as follows:

First, it does an inward clocking of all the parties that are not actively corrupted

Then it interacts with: Authenticated Transfer (Ideal Functionality)

Then, it does an outward clocking of all the parties that are not actively corrupted



We say that the composed party is inward clocked when:

Z first clocks Q1, then P1, then Z sends $(\text{inclock}, i)$ on AT.infl and F_{AT} clocks

We say that the composed party is outward clocked when:

Z first sends $(\text{outclock}, i)$ on AT.infl and F_{AT} then clocks P1 and then Q1 clocks

This allows F_{AT} to send messages to the composed parties.

Synchronous Communication

If n parties, $P_1, P_2, \dots, P_n$ are using F_{AT} as a communication resource

Then, even though the parties are synchronized, the communication is not.

Why?

F_{AT} is not modeled as a synchronous resource.

How do we turn F_{AT} to a synchronous resource?

On (inclock, P_i) process all messages on $AT.in_i$ as F_{AT} does

On (outclock, P_i) for all stored values (mid, j, i, m) delete (mid, j, i, m) and outputs (mid, j, m) on $AT.out_j$

Complete Breakdown

We say that the ideal functionality F_F

does a complete breakdown, we mean that it starts behaving as if all parties are actively corrupted.

Models worst-case failure (all parties corrupted)

Functionality stops enforcing security

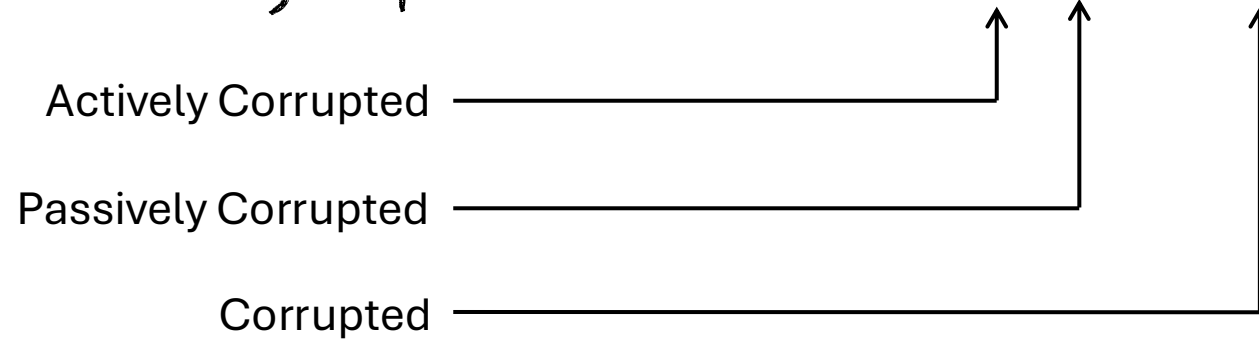
Leaks full internal state on $F.\text{leak}$

On (send, i, m) : outputs m to party P_i

On input m from P_i : leaks (i, m)

Secure Synchronous Communication and Broadcast

Ideal Functionality keeps track of three sets: A, P, and C



What will be the output on SC.leak?

Honest Inputs Parse the message on $SC.in_i$: $m_{i,1}, \dots, m_{i,n}, m_i$

$\{m_{i,j}\}_{j \in C} m_i$

$m_{i,j} \longrightarrow$ The message P_i is send to P_j

$m_i \longrightarrow$ The message P_i is broadcasting

Secure Synchronous Communication and Broadcast

Corrupted Inputs

(change, i , $(m_{i,1}, \dots, m_{i,n}, m_i)$)

where, $i \in A$ and at a point before which the clock-out phase has started,

Store $((m_{i,1}, \dots, m_{i,n}, m_i))$

Overriding any previously stored input from the corrupted party i in this round

(As long as any passively corrupted or honest party has received outputs, actively corrupted parties can change their outputs)

Delivery

On input (clockout, i) on SC.infl

If there is any party for which $m_{i,1}, \dots, m_{i,n}, m_i$ is not stored, then store

$$m_{i,1}, \dots, m_{i,n}, m_i = \epsilon, \dots, \epsilon$$

Then: $(m_{i,1}, \dots, m_{i,n}, m_1, \dots, m_n) = SC.out_i$

Secure Synchronous Function Evaluation

Initialize:

`delivery-round ← evaluated inputs-ready`
`input-read1, ..., input-readn ∈ {0, 1}`



Honest Inputs: (clockin, on SFE.infl. (Trigger!))

i)

`i ∉ A (honest)`

IF: `input  $x_i$  is received`
`input-readyi = 0`

THEN:

`input-readyi ← 1`
`Store (i,  $x_i$ )`
`Leak (input,`
`i)`

Secure Synchronous Function Evaluation

Corrupted Inputs: (change, i , x_i) where $i \notin A$ (corrupted)

If (evaluated == 0)

Set input-read $\leftarrow 1$

Store: (i , x_i) overriding any previously stored for party i

Simultaneous Inputs: If it holds in some round, then after the clock-in phase ends, there exists $i, j \notin A$

such that: input-read $_i \leftarrow 0$
input-read $_j \leftarrow 1$

Then do a complete breakdown

If it happens in some round that after the clockin phase ends,

input-read $_i \leftarrow 1$
 $i \notin A$
Inputs-ready $\leftarrow 0$

Secure Synchronous Function Evaluation

Evaluate Function: Trigger: (evaluate)

IF:

inputs-ready = 1
(evaluated == 0)

THEN:

Set: evaluated
 $\leftarrow 1$
Compute: $(y_1, \dots, y_n = f(x_1, \dots, x_n))$
Leak outputs for parties in C

Secure Synchronous Function Evaluation

Simultaneous Output: On input delivery-round on `SFE.infl`, where:

```
evaluated = 1 and delivery-round=0,
```

Proceed as follows:

If we are at a point where no party $i \notin A$ was *clocked-out* yet, then set:

delivery-round=1

Delivery: Trigger: (clockout, i)

IF: `delivery-round =`

Then: ¹ Output y_i to party i